

SOEN 240 – Object Oriented Programming I (Java)

Lecturer: Dr. Okemwa | Kisii University | Clean Revision Notes

1. Programming Languages & Basic Concepts

A **programming language** is an artificial language used to express computations that can be performed by a machine. It consists of words and symbols that are used to write a computer program.

Key Definitions

Syntax

A set of rules that determine how symbols can be put together to form a valid program statement. Some languages are better for one type of program than others. Syntax rules define how we can put together symbols, reserved words and identifiers to make a valid program.

Semantics

The meaning of a program statement. Semantics define what the statement means (its purpose/role in a program). A program that is syntactically correct is not always logically (semantically) correct. A program always does what we tell it to do — not what we meant it to do.

Program Translation

Source code (human-readable) must be translated into machine language before it can be executed. Two common tools are used:

- **Compiler** – A software tool that translates source code into a specific target language (usually machine language for a particular CPU type).
- **Interpreter** – Translates and executes source code line by line (or statement by statement).

Java special case

Java bytecode is **not** machine language for any particular CPU. An Interpreter (the Java Virtual Machine) translates bytecode into machine language and executes it. This implies that a Java program is not tied to any particular machine — Java is considered **architecture neutral**.

2. History and Properties of the Java Language

Java was created by **James Gosling** at Sun Microsystems. It was introduced in **1995** and has grown quickly in popularity as a high-level programming language.

Important Properties of Java

- **Simple** – Easier to learn and use than many older languages.
- **Object Oriented** – Everything is built around objects and classes.
- **Familiar** – Syntax is similar to C/C++, so programmers from those languages adapt quickly.
- **Robust** – Strong memory management and error handling.
- **Secure** – Designed with security features (sandbox, bytecode verification, etc.).
- **Architecture Neutral** – Bytecode runs on any machine that has a JVM.
- **Portable** – Write once, run anywhere.
- **Interpreted** – Bytecode is interpreted (or JIT-compiled) by the JVM.
- **High Performance** – Modern JVMs use Just-In-Time compilation for speed.
- **Multithreaded** – Built-in support for concurrent programming.
- **Dynamic** – Classes can be loaded at runtime.
- **Distributed** – Good support for networked applications.

Java is also automatic garbage collected — memory that is no longer referenced is automatically reclaimed by the JVM.

3. Java Development Environment

There are many programs (IDEs and tools) used to develop Java software. Common ones include:

- JDK (Java Development Kit) – the core tools (compiler, JVM, libraries)
- NetBeans
- Eclipse
- IntelliJ IDEA / Android Studio
- Borland JBuilder (older)

You can also write Java source in any text editor and compile it from the command line using **javac** and run it with **java**.

4. Structure of a Java Program

A typical Java program contains:

- Package declaration (optional but recommended)
- Import statements
- Class definition(s)
- Methods (including the special **main** method for applications)
- Statements and expressions

Every Java application must have a **main** method — this is the entry point of the program. Applets and some other types of programs follow different rules.

Comments

- Single-line: `// this is a comment`
- Multi-line: `/* comment block */`
- Documentation comments: `/** ... */` (used by javadoc)

5. Classes and Objects (Core OOP Ideas)

Java is built around the concepts of **classes** and **objects**.

Class

A blueprint or template that describes the data (attributes) and behaviour (methods) that objects of that type will have. A class is a collection of related Java methods and variables.

Object

An instance of a class. When you create an object, you get a concrete realisation of the class blueprint.

Four pillars of Object-Oriented Programming that Java supports:

- **Encapsulation** – bundling data and methods that operate on that data, and controlling access.
- **Inheritance** – a class can inherit attributes and methods from another class.
- **Polymorphism** – the ability of different objects to respond to the same message in different ways.
- **Abstraction** – hiding complex implementation details and showing only the necessary features.

6. Access Modifiers & Class Members

Access modifiers control who can see and use a class, method or variable:

- **public** – accessible from anywhere.
- **private** – accessible only inside the same class.
- **protected** – accessible inside the same package and by subclasses.
- **default** (no modifier) – accessible only inside the same package.

A class body contains variables (fields) and methods. You can declare variables inside a class (instance or static variables) or inside a method (local variables).

7. Variables and Data Types

A variable is a named storage location in memory. In Java you must declare a variable before you use it.

Primitive data types (built-in):

- Integer types: byte, short, int, long
- Floating-point: float, double
- Character: char
- Boolean: boolean (true / false)

Reference types:

Objects, arrays, and Strings are reference types. A variable of a reference type stores a reference (address) to the actual object in memory, not the object itself.

Example declaration and assignment:

```
int age = 20;
String name = "Hello";
double price = 99.50;
```

8. Methods

A method is a block of code that performs a specific task. Methods help organise a program into logical pieces.

A method has:

- A name
- Optional parameters (arguments with their types)
- A return type (or **void** if it returns nothing)
- A body containing statements

You can declare variables inside a method — these are local variables and exist only while the method is executing.

The special method **main** is the starting point of a Java application:

```
public static void main(String[] args) { ... }
```

9. Packages and Reserved Words

Package

A package is a way of grouping related classes. It helps avoid name conflicts and makes large projects easier to manage. You declare a package at the top of a source file with the **package** keyword.

Reserved words (keywords)

Words that have special meaning in Java and cannot be used as names for variables, methods or classes. Examples: class, public, static, void, int, if, else, while, for, return, new, this, super, etc.

10. Quick Revision Checklist

- Java source code is compiled into bytecode, then executed by the JVM.
- Java is architecture-neutral and portable (“write once, run anywhere”).
- Syntax = rules of structure; Semantics = meaning of the statements.
- A class is a blueprint; an object is an instance of a class.
- Four OOP pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.
- Access modifiers: public, private, protected, default.
- Every application needs a main method as the entry point.
- Primitive types store values; reference types store addresses of objects.

These notes are based on class lectures by Dr. Okemwa (SOEN 240). Use them for quick revision before CATs and exams. Expand with your own examples for deeper understanding.